ISA 401: Business Intelligence & Data Visualization

07: Data Wrangling with dplyr

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

@Fadel Megahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Fall 2026

Before We Start

1 Join today's Zoom

Use the Zoom link in the **Week 0 Module** on Canvas. **Mute** your mic and **turn off** your video.
Log in from the **lab's computer** so you can **share your screen** if you need help with an error.

2 Open the project

GitHub Desktop → `business_intelligence` → **Fetch origin**. Then double-click **business_intelligence.Rproj**.
Top right corner of RStudio must say `business_intelligence` before you type anything.

3 Start today's markdown

File → New File → R Markdown..., title *Class 07*, **OK**. Delete everything below the setup chunk, then save as **markdowns/class07.Rmd**.
Same as Monday. Your `class06.Rmd` stays as it is.

4 Get the app into your repo

Download **job_scout_chat.zip** from the Week 3 module and unzip it into your repo as **apps/job_scout_chat/**.
Part 1 starts by running this app on your PC, so have the folder in place before we begin.

Done early? **Help the person next to you.** Stuck? **Share your screen on Zoom** and we fix it together.

Learning Objectives for Today's Class

- **Run** the `querychat` app locally and **deploy** it to Render from your GitHub repository, with the API key kept out of every commit.
- **Manipulate** a data frame with `dplyr::select()`, `dplyr::filter()`, `dplyr::arrange()`, `dplyr::mutate()`, and `dplyr::across()`, chained with `|>`.
- **Summarize** groups with `dplyr::group_by()` and `dplyr::summarize()`, and **count** with `dplyr::count()`.
- **Combine** two tables with `dplyr::left_join()`, and say what `dplyr::inner_join()` and `dplyr::anti_join()` would have dropped.

Part 1 20 minutes

Running and deploying the app. The app on your PC, the leaked key and what it changes, why the Dockerfile exists, the deploy on the projector. Builds run while we move on.

Part 2 50 minutes

Data wrangling with dplyr. From SQL to dplyr, the verbs one question at a time with a small data frame before and after each, selection helpers and across(), joins, then ten minutes of practice on the Airbnb listings from Assignment 04.

Part 1: Running and Deploying the App

Running the App Locally

```
# apps/job_scout_chat/app.R
library(querychat)

con = DBI::dbConnect(RSQLite::SQLite(), "data/scout.db")

client = ellmer::chat_openai(
  model  = "gpt-5.6-luna",
  params = ellmer::params(reasoning_effort = "none")
)

qc = querychat(
  con, "scout_postings",
  client   = client,
  tools    = c("filter", "query", "visualize"),
  greeting = "Ask me about the 1,891 job postings
              ChatISA Job Scout collected."
)

qc$app_obj()
```

The path is `"data/scout.db"`, not `"../data/"`: an app runs from **its own folder**, so the app gets its own copy of the data.

Four steps

1. You unzipped `job_scout_chat.zip` into `apps/job_scout_chat/` before class. It holds `app.R`, the `Dockerfile`, `README.md`, and `data/scout.db`.

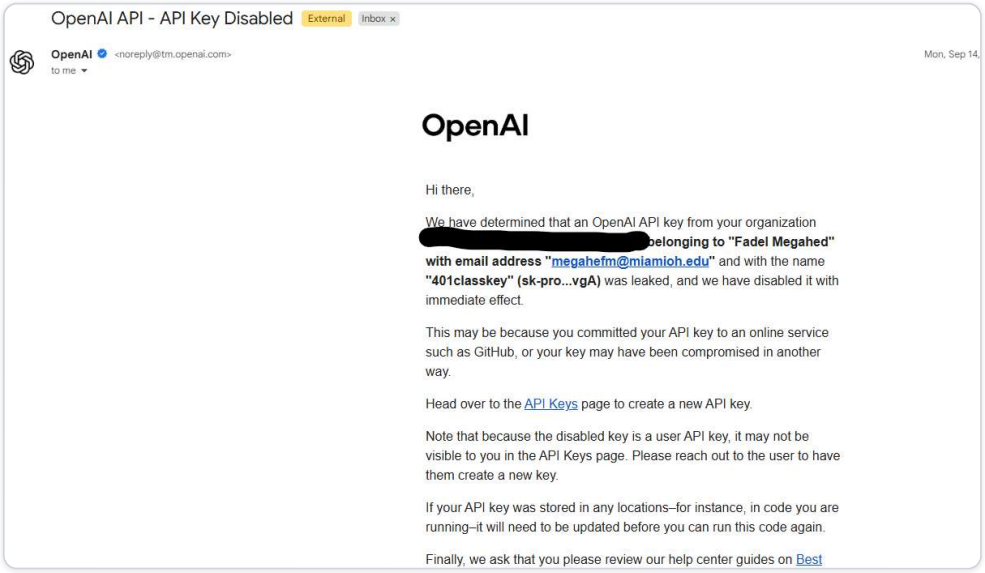
2. Open `app.R` from that folder. It is the code on the left; type it yourself if you prefer.

3. In the Console: `install.packages("querychat", dependencies = TRUE)` (pulls in `DT` and `ggsql`, which the app needs but querychat only suggests), then `usethis::edit_r_environ("project")`, add the line `OPENAI_API_KEY=` followed by the **new class key** (Canvas; the old one is dead, next slide), save. Then `usethis::use_git_ignore(".Renviron")`, and **Session → Restart R**.

4. Click **Run App** at the top of `app.R`.

The key never goes in a file you commit. `.Renviron` from step 3 sits next to your `.Rproj`, so it follows your M: drive to any lab PC; the `use_git_ignore()` line keeps it out of every commit.

The Leaked API Key



OpenAI API - API Key Disabled External Inbox ×

OpenAI <noreply@tm.openai.com> Mon, Sep 14,
to me

OpenAI

Hi there,

We have determined that an OpenAI API key from your organization [redacted] belonging to "Fadel Megahed" with email address "megahefm@miamioh.edu" and with the name "401classkey" (sk-pro...vgA) was leaked, and we have disabled it with immediate effect.

This may be because you committed your API key to an online service such as GitHub, or your key may have been compromised in another way.

Head over to the API Keys page to create a new API key.

Note that because the disabled key is a user API key, it may not be visible to you in the API Keys page. Please reach out to the user to have them create a new key.

If your API key was stored in any locations–for instance, in code you are running–it will need to be updated before you can run this code again.

Finally, we ask that you please review our help center guides on Best

The email that arrived Monday, Sep 14. Organization name redacted; the key is shown by its first and last characters only, which is how OpenAI refers to keys.

What happened

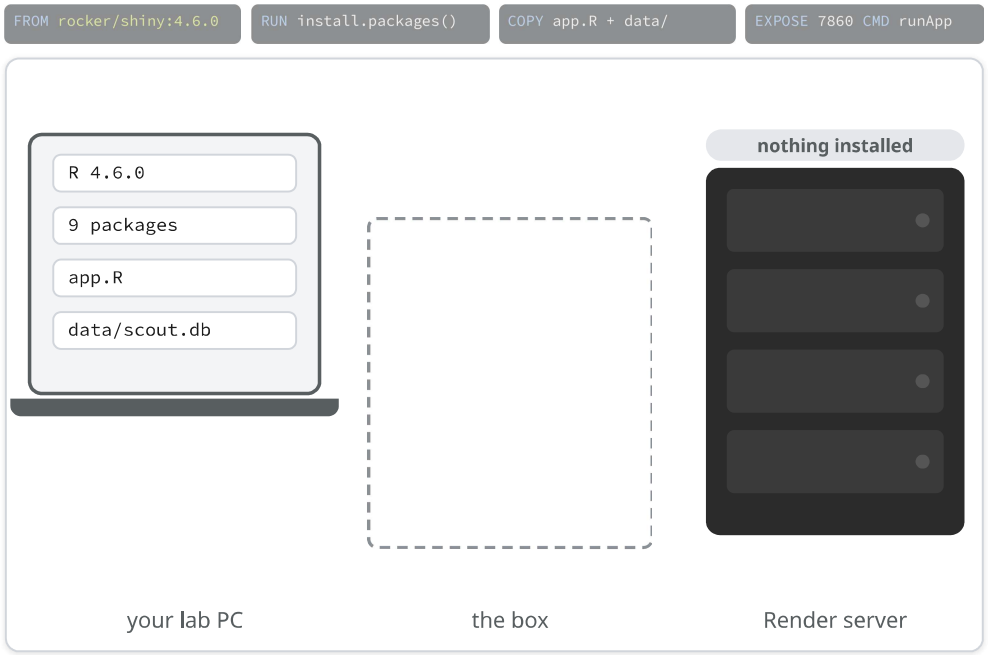
A file containing the class key was pushed to a public GitHub repository. Automated scanners watch every public commit for strings that look like keys. OpenAI was told, and disabled the key **the same day**, before anyone could use it.

What it means

A key in a commit is public within minutes, even if you delete the file in the next commit: the history keeps it. There is no un-leaking; the only fix is to **revoke** the key, make a new one, and replace it **everywhere it was used**.

What you do now. The **new class key** goes in your project `.Renviron`; then **Session → Restart R**. If you deployed the app on Monday, replace its environment variable too. Nobody is in trouble: this is exactly why the `use_git_ignore()` line exists.

Why a Dockerfile? Ship the Computer



Before the recipe. Your app runs on this PC only because R, nine packages, app.R, and scout.db all happen to be here. The server on the right has none of them.

Back | Step | Play | Reset | 0.5x | 1x | 2x

0 / 6

Why it matters to you

In the **1,891** Job Scout postings, the description mentions:

- **351** deploy or deployment
- **249** AWS, Azure, or Google Cloud
- **31** Docker
- **31** Kubernetes

Deploying is a common ask. Docker itself is named only in machine-learning and data-engineering roles: recognize it, do not fear it.

Industry-wide

71.1% of the 24,473 developers who answered the Cloud development question in the Stack Overflow 2025 Developer Survey use Docker. A Dockerfile is a recipe for a computer; Hugging Face reads the recipe, so nobody installs anything by hand.

Three words from job postings: **recipe** (Dockerfile) → **box** (image) → **running box** (container).

Sources: `data/scout.db` (ChatISA Job Scout harvest through 2026-08-23; counts are case-insensitive matches in the posting text); Stack Overflow 2025 Developer Survey, Technology, Cloud development; Docker: What is a container.

Deploy to Render

1 Account

Sign up at **render.com** with **GitHub**, using the account that owns `business_intelligence`. The Hobby plan is free and asks for no card. Skip the onboarding questions.

2 Connect the repository

New → Web Service → Git Provider → GitHub. Install Render's GitHub app on `business_intelligence` (*Only select repositories*), then click the repository to connect it. Your app folder is already in it from the ritual.

3 Configure

Name `job-scout-chat`. Language **Docker** (detected from the Dockerfile). Branch `main`. **Root Directory** `apps/job_scout_chat`. Instance type **Free**, $0 (the default is paid; change it). Environment variable `OPENAI_API_KEY` = the **class key**. **Deploy Web Service**.

4 Watch it build

Logs run the Dockerfile line by line; a few minutes the first time. Status **Live**, address `https://job-scout-chat-xxxx.onrender.com`. A free app sleeps after 15 idle minutes and wakes in about a minute; a 502 while waking is normal.

What the Dockerfile is. A recipe that builds a fresh Linux computer with the same R version as this lab, installs the packages from a dated snapshot, copies your app in, and starts it. You do not run Docker; Render does. Same variable name as the line in your `.Renviron`: `app.R` reads `OPENAI_API_KEY` from wherever it runs, so the key is never in a file you commit. The recipe is read line by line in Appendix A1 of the Class 06 deck.

Part 2: Data Wrangling with dplyr

From SQL to dplyr

Which companies posted the most remote jobs? Monday you typed it in SQL, then watched the app write the same SQL from English. Today you write it in R, and the answer does not change.

Copy

```
# Monday, Part 1: SQL, typed by you
DBI::dbGetQuery(con, "
  SELECT company, COUNT(*) AS n
  FROM scout_postings
  WHERE remote = 1
  GROUP BY company
  ORDER BY n DESC
  LIMIT 10")
```

Copy

```
# Today: dplyr, on the data frame from dbReadTable()
postings |>
  dplyr::filter(remote == 1) |>
  dplyr::count(company, sort = TRUE) |>
  head(10)
```

Read the pipe as "then"

Take `postings`, **then** keep the remote rows, **then** count per company, **then** the first ten. Each line takes a data frame in and hands a data frame to the next line. `|>` is base R; type it with **Ctrl+Shift+M** once Tools → Global Options → Code has "native pipe" checked.

Same verbs, R spelling

`WHERE`	`dplyr::filter()`
`GROUP BY` + `COUNT(*)` + `ORDER BY`	`dplyr::count(sort = TRUE)`
`LIMIT`	`head()`, base R

Same notation as every class: `dplyr::verb()` for every dplyr function, so a reader always knows which package a verb came from. Base R functions such as `head()` and `as.Date()` stay bare.

The dplyr Verbs

Verb	What it changes	Example	SQL cousin
`select()`	**Columns.** Keep some, drop some, reorder.	`dplyr::select(company, title, remote)`	`SELECT`
`filter()`	**Rows.** Keep the rows where a test is TRUE.	`dplyr::filter(remote == 1, location_state == "OH")`	`WHERE`
`arrange()`	**Order.** Sort by one or more columns; `desc()` for largest first.	`dplyr::arrange(dplyr::desc(posted_at))`	`ORDER BY`
`mutate()`	**New columns** computed from old ones; the row count never changes.	`dplyr::mutate(days_open = last_seen - posted)`	`SELECT ... AS`
`group_by()` + `summarize()`	**One row per group.** `n()` counts, `mean()` averages; `count()` is the shortcut.	`dplyr::group_by(company) |> dplyr::summarize(n = dplyr::n())`	`GROUP BY`
`left_join()`	**Two tables, one key.** Add columns from a second table where the key matches.	`dplyr::left_join(states, by = "location_state")`	`JOIN`

Why the pipe works: every verb takes a data frame as its first argument and returns a data frame. That is why `|>` can hand one verb's result to the next, and why a chain never needs a temporary variable. The original `postings` is never modified; a verb returns a new data frame, and you keep it only if you assign it.

A dplyr Pipeline, Step by Step

jobs → select() → filter() → mutate() → arrange() → group_by() → summarize()

jobs: eight of the 1,891 rows, seven columns. The same eight rows as Monday's animation.

company	title	location_state	remote	category	posted_at	last_seen_at
The Home Depot	Data Scientist - People	GA	1	fulltime	2026-08-11	2026-08-16
CVS Health	Data Engineer	IN	1	fulltime	2026-08-14	2026-08-16
OhioHealth	Requirement Business Ana	OH	0	fulltime	2026-08-14	2026-08-16
The Home Depot	Cybersecurity Analyst II	TX	1	fulltime	2026-08-07	2026-08-09
Talkspace	Data Analyst	NY	1	fulltime	2026-08-19	2026-08-23
The Home Depot	Business Analyst	TX	0	fulltime	2026-08-21	2026-08-23
CVS Health	Data Scientist (Decision	IN	1	fulltime	2026-07-31	2026-08-02
The Home Depot	Data Scientist, Pricing	GA	1	fulltime	2026-08-19	2026-08-23

```r
jobs |>
  dplyr::select(company, location_state, remote, posted_at, last_seen_at) |>
  dplyr::filter(remote == 1) |>
  dplyr::mutate(days_open = as.numeric(as.Date(last_seen_at) - as.Date(posted_at))) |>
  dplyr::arrange(dplyr::desc(days_open)) |>
  dplyr::group_by(company) |>
  dplyr::summarize(n = dplyr::n(), avg_days = mean(days_open))
```

Start. A data frame goes in at the left. Each station is one verb; each verb hands a data frame to the next.

Back | Step | Play | Reset | 0.5x | 1x | 2x

0 / 6

12 / 32

select(): Choosing Columns

Question. `scout_postings` has 19 columns. Today's questions need seven of them. How do we keep only those?

postings: 5 of 1,891 rows, 5 of 19 columns

company	~~title~~	location_state	remote	~~category~~
The Home Depot	~~Data Scientist~~	GA	1	~~fulltime~~
CVS Health	~~Data Engineer~~	IN	1	~~fulltime~~
OhioHealth	~~Requirement Bu~~	OH	0	~~fulltime~~
The Home Depot	~~Cybersecurity~~	TX	1	~~fulltime~~
Talkspace	~~Data Analyst~~	NY	1	~~fulltime~~

dplyr::select(company, location_state, remote) →

3 columns, the same 5 rows

company	location_state	remote
The Home Depot	GA	1
CVS Health	IN	1
OhioHealth	OH	0
The Home Depot	TX	1
Talkspace	NY	1

Columns only Name the columns to keep, in the order you want them. Every row survives. `dplyr::select(-title)` drops instead; `company:remote` takes a range. The result is a new data frame; `postings` is unchanged unless you assign over it.

Copy

```
con = DBI::dbConnect(RSQLite::SQLite(), "../data/scout.db")
postings = DBI::dbReadTable(con, "scout_postings") |> tibble::as_tibble()
DBI::dbDisconnect(con)

jobs = postings |>
  dplyr::select(company, title, location_state, remote, category, posted_at, last_seen_at)
jobs
```

```
# A tibble: 1,891 × 7
  company    title location…¹ remote category posted_at last_see…²
  <chr>      <chr> <chr>       <int> <chr>    <chr>     <chr>
1 UnitedHea… Data… GA              0 fulltime 2026-07-… 2026-07-2…
2 Anza Mort… Busi… NC              0 fulltime 2026-07-… 2026-08-0…
3 CoServ     Busi… TX              0 fulltime 2026-07-… 2026-07-2…
# ℹ 1,888 more rows
# ℹ abbreviated names: ¹location_state, ²last_seen_at
```

filter(): Choosing Rows

Question. Which postings are remote? And which are remote *and* in Ohio?

jobs: 5 of the rows

company	location_state	remote
The Home Depot	GA	1
CVS Health	IN	1
~~OhioHealth~~	~~OH~~	~~0~~
The Home Depot	TX	1
Talkspace	NY	1

`dplyr::filter(remote == 1)` →

the rows where the test is TRUE

company	location_state	remote
The Home Depot	GA	1
CVS Health	IN	1
The Home Depot	TX	1
Talkspace	NY	1

Rows only One test per row; TRUE stays, FALSE leaves; the columns are untouched. Two tests separated by a comma must both be TRUE; `|` means either; `%in%` tests membership in a set; `is.na()` tests for an empty cell. Two equals signs compare; one assigns. Text goes in quotes, column names do not.

Copy

```
c(one_test = jobs |> dplyr::filter(remote == 1) |> nrow(),
  both     = jobs |> dplyr::filter(remote == 1, location_state == "OH") |> nrow(),
  either   = jobs |> dplyr::filter(remote == 1 | category == "internship") |> nrow(),
  in_set   = jobs |> dplyr::filter(location_state %in% c("OH", "IN", "KY")) |> nrow())
```

```
one_test     both   either   in_set
     127       18      212      212
```

mutate() and arrange()

Question. How many days did each posting stay up, and which stayed up longest? No column says; two columns together do.

jobs

company	posted_at	last_seen_at
The Home Depot	2026-08-11	2026-08-16
CVS Health	2026-08-14	2026-08-16
OhioHealth	2026-08-14	2026-08-16
The Home Depot	2026-08-07	2026-08-09
Talkspace	2026-08-19	2026-08-23

```
dplyr::mutate(days_open = ...) |>
dplyr::arrange(dplyr::desc(days_open))
```

→

a new column, then sorted by it

company	posted_at	last_seen_at	days_open
The Home Depot	2026-08-11	2026-08-16	5
Talkspace	2026-08-19	2026-08-23	4
CVS Health	2026-08-14	2026-08-16	2
OhioHealth	2026-08-14	2026-08-16	2
The Home Depot	2026-08-07	2026-08-09	2

A new column, then a new order `dplyr::mutate()` computes `new = expression` once per row from that row's cells; the row count never changes. `as.Date()` turns text into real dates so subtraction gives days. `dplyr::arrange()` sorts, smallest first unless wrapped in `dplyr::desc()`; a second column breaks ties.

Copy

```
jobs = jobs |>
  dplyr::mutate(days_open = as.numeric(as.Date(last_seen_at) - as.Date(posted_at)))

jobs |> dplyr::arrange(dplyr::desc(days_open)) |> dplyr::select(company, title, days_open)
```

```
# A tibble: 1,891 × 3
  company                       title                   days_open
  <chr>                         <chr>                       <dbl>
1 Veterans Health Administration Clinical Laboratory Sc…        30
2 Buxton                        Data Analyst                    30
3 HHC                           Business Intelligence …         30
# ℹ 1,888 more rows
```

Selection Helpers and across()

Question. Both date columns are text. Convert both to dates without writing each name twice, and later: the median of every numeric column, per city.

two text columns that end in _at

company	posted_at	last_seen_at
The Home Depot	2026-08-11	2026-08-16
CVS Health	2026-08-14	2026-08-16
OhioHealth	2026-08-14	2026-08-16
The Home Depot	2026-08-07	2026-08-09
Talkspace	2026-08-19	2026-08-23

```
dplyr::mutate(dplyr::across(
dplyr::ends_with("_at"), as.Date))
```

→

both converted to Date

company	posted_at	last_seen_at
The Home Depot	2026-08-11	2026-08-16
CVS Health	2026-08-14	2026-08-16
OhioHealth	2026-08-14	2026-08-16
The Home Depot	2026-08-07	2026-08-09
Talkspace	2026-08-19	2026-08-23

Pick columns by rule, then apply one function to all of them

`dplyr::starts_with("location")`	names that begin with the text; `dplyr::ends_with()` and `dplyr::contains()` work the same way
`dplyr::any_of(c("price", "beds"))`	these names, if present; no error when one is missing
`dplyr::where(is.numeric)`	every column of a type; `dplyr::everything()` is all remaining columns
`dplyr::across(dplyr::ends_with("_at"), as.Date)`	inside `mutate()` or `summarize()`: one function applied to every matching column
`dplyr::if_any(c(title, company), is.na)`	inside `filter()`: one test over several columns

Copy

```
jobs = jobs |> dplyr::mutate(dplyr::across(dplyr::ends_with("_at"), as.Date))    # both dates at once
jobs |> dplyr::summarize(dplyr::across(dplyr::where(is.numeric), mean))
```

```
# A tibble: 1 × 2
  remote days_open
   <dbl>     <dbl>
1 0.0672      4.21
```

group_by() and summarize()

Question. How many postings per state, and what share of them are remote? One number per state, not one per posting.

jobs: 5 rows, 3 companies

company	location_state	remote
The Home Depot	GA	1
CVS Health	IN	1
OhioHealth	OH	0
The Home Depot	TX	1
The Home Depot	TX	0

```
dplyr::group_by(company) |>
  dplyr::summarize(...)
```



one row per company

company	n	remote_share
The Home Depot	3	0.67
CVS Health	1	1
OhioHealth	1	0

Label the groups, then collapse each to one row `dplyr::group_by()` only marks which rows belong together; `dplyr::summarize()` collapses each group to one row with the grouping column plus what you name (`dplyr::n()`, `mean()`, `median()`, `sum()`). `dplyr::count(company, sort = TRUE)` is the shortcut for a sorted count alone: Monday's query in one line.

Copy

```
jobs |>
  dplyr::group_by(location_state) |>
  dplyr::summarize(postings = dplyr::n(), remote_share = mean(remote)) |>
  dplyr::arrange(dplyr::desc(postings))
```

```
# A tibble: 47 × 3
  location_state postings remote_share
  <chr>             <int>        <dbl>
1 CA                  328       0.0457
2 TX                  281       0.0890
3 NY                  183       0.0874
# ℹ 44 more rows
```

Demo: Remote Work in the Postings

Question. Which postings are remote, and does it depend on the job category, the state, or the kind of role? Typed live; you say the verb before I type it.

```
con = DBI::dbConnect(RSQLite::SQLite(), "../data/scout.db")
postings = DBI::dbReadTable(con, "scout_postings") |> tibble::as_tibble()
DBI::dbDisconnect(con)

jobs = postings |>
  dplyr::select(company, title, location_state, remote, category, posted_at, last_seen_at) |>
  dplyr::mutate(dplyr::across(dplyr::ends_with("_at"), as.Date),
                days_open = as.numeric(last_seen_at - posted_at))
```

1 By category 2 By state 3 By role

Which category is remote-friendly? One verb pair. The federal postings come from USAJobs and are on site by policy; internships almost never; so the remote question is a full-time question.

```
# A tibble: 3 × 3
  category   postings remote_share
  <chr>         <int>        <dbl>
1 federal         548        0
2 fulltime       1257        0.1
3 internship       86        0.012
```

Joins

by_state

location_state	postings
CA	328
TX	281
OH	120
DC	58
NA	30

states

location_state	state_name	region
CA	California	West
TX	Texas	South
OH	Ohio	North Central
NY	New York	Northeast
GA	Georgia	South

```
# the key: location_state holds the same values, meaning the same thing, in both tables
by_state |> dplyr::left_join(states, by = "location_state")    # keep every row on the left
by_state |> dplyr::inner_join(states, by = "location_state")   # keep only the matches
by_state |> dplyr::anti_join(states, by = "location_state")    # keep only the non-matches
```

Two tables. Left: postings per state, five of the 47 rows. Right: a lookup built from R's own `state.abb`, `state.name`, `state.region`. Neither knows about the other yet.

Back | **Step** | Play | Reset | 0.5x | 1x | 2x

0 / 4

Joins in Practice

```
states = tibble::tibble(location_state = state.abb,
                        state_name = state.name,
                        region = as.character(state.region))

jobs |>
  dplyr::left_join(states, by = "location_state") |>
  dplyr::group_by(region) |>
  dplyr::summarize(postings = dplyr::n(), remote_share = mean(remote)) |>
  dplyr::arrange(dplyr::desc(postings))
```

```
# A tibble: 5 × 3
  region        postings remote_share
  <chr>            <int>        <dbl>
1 South              711       0.0675
2 West               420       0.0357
3 North Central      373       0.0965
# ℹ 2 more rows
```

```
jobs |> dplyr::count(location_state) |> dplyr::anti_join(states, by = "location_state")
```

```
# A tibble: 2 × 2
  location_state     n
  <chr>          <int>
1 DC                58
2 <NA>              30
```

A question the database alone cannot answer

`scout.db` has state codes and nothing else about geography. Four lines add a region to every posting and count by it. The `NA` row is DC plus the postings with no state; a left join keeps them and says so.

The join family

`dplyr::left_join()`	all left rows; NA where no match
`dplyr::inner_join()`	matches only
`dplyr::anti_join()`	left rows with no match
`dplyr::full_join()`	everything from both sides

When the key has different names, `by = c("location_state" = "abb")`.

After any join that matters, run `dplyr::anti_join()` once. If it returns rows you did not expect, the key is wrong or the lookup is incomplete.

Practice: Airbnb Listings

05:00

Task Answer Discussion

```r
con = DBI::dbConnect(RSQLite::SQLite(), "../data/midwest_airbnb.db")
listings = DBI::dbReadTable(con, "listings") |>
  tibble::as_tibble()
DBI::dbDisconnect(con)
dplyr::glimpse(listings)
```

Four questions on 14,887 listings

1. How many listings per `city`?
2. For entire homes (`room_type == "Entire home/apt"`) that sleep four or more, what is the **median price** in each city?
3. Add `price_per_person = price / accommodates`. Which three listings are cheapest per person? Believe them?
4. Which five `host_name`s have the most listings?

What Comes Next

Monday: APIs

An API answers in JSON; `fromJSON()` turns it into a data frame; the verbs do the rest. The Cincinnati crash feed from Class 04 becomes `dplyr::count(day)` and `dplyr::arrange()` instead of a bracket chain.

Sep 23 to 30: scraping

A scraped HTML table arrives messy: text where numbers should be, one table per page. `dplyr::mutate()` cleans, and a join or `dplyr::bind_rows()` stacks pages. Every scraping day ends in a dplyr chain.

Assignment 05: the dictionary

`dplyr::count()` a column to learn its values; `dplyr::summarize(min(), max())` for its range; `dplyr::filter(is.na())` for its gaps. That is how you write the 23 missing rows of `data_desc.md`.

And the app you deployed today has been writing these for you all along: `WHERE` is `dplyr::filter()`, `GROUP BY` is `dplyr::group_by()` + `dplyr::summarize()`, `ORDER BY` is `dplyr::arrange()`. When you read its SQL, you are reading a dplyr chain in another spelling.

Our Class Git + GitHub Ritual

1 Knit

Click **Knit**. `class07.html` re-runs every chunk top to bottom and saves the page next to your markdown.

If the knit fails, a chunk depends on something you only ran in the Console. Fix the markdown, not the Console.

→

2 Commit

GitHub Desktop lists today's work: `markdowns/class07.Rmd`, `class07.html`, `apps/job_scout_chat/`, and `.gitignore` with its new `.Renviron` line.

If `.Renviron` itself is in the list, stop: uncheck it and redo the `use_git_ignore()` step. Summary: `Add class 07 markdown and the job scout app`.

→

3 Push and verify

Push origin, then check the repo on GitHub: `markdowns/` and `data/` both there, readable from any device.

That is code, data, and output, together.

If it is not pushed, it is not done. Your repo now carries code, data, and output that anyone, including future you, can re-run.

Recap

Summary of Main Points

By now, you should be able to do the following:

- **Run** the `querychat` app locally with a git-ignored `.Renviron` and **deploy** it to Render with the key as an **environment variable**; a key in a commit is public within minutes.
- **Chain verbs with `|>`**: `dplyr::select()` columns, `dplyr::filter()` rows, `dplyr::arrange()` order, `dplyr::mutate()` new columns, `dplyr::across()` for many columns at once.
- **Collapse** with `dplyr::group_by()` + `dplyr::summarize()` or `dplyr::count()`; **join** with `dplyr::left_join()`, then `dplyr::anti_join()` for what did not match.

Reading for Assignment 05: Class 06 appendices A1 to A4, plus today's A2 deploy troubleshooting. **Next class (Mon, Sep 21):** APIs, where these verbs take over.

Before You Go

Join at **https://www.menti.com/al37nbpvnzpg**

Before you go: one thing from today you w

Review and Clarification

- **Class Notes**: Take some time to revisit your class notes for key insights/concepts.
- **Zoom Recording**: The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **AI Recaps:** make your own with Gemini's NotebookLM, as I did for Class 01 (see the right).
- **Questions**: Do not let confusion pile up.

Make your own recap in NotebookLM

- Go to notebooklm.google.com and sign in with your Miami Google account.
- Add today's slide deck as a source: paste the deck's address from Canvas as a **Website** source.
- Click **Audio Overview** for a podcast, or **Video Overview** for a narrated video. Both take a few minutes to generate.

What it looks like: the Class 01 video and podcast. Treat them as a study aid, not a substitute for the slides: the AI summarizes what is on the page and can misstate details.

🎯 Assignments 🎯

Assignment 05: Deploy Your Midwest Airbnb Chat

Today's recipe on the Airbnb listings you just explored: same Dockerfile, new data, a full data dictionary, extra instructions, a theme and an About section of your own (the `page_sidebar()` code in Class 06, Appendix A3 is enough). Deployed from your GitHub repository, three or more commits. Submit the folder's GitHub address; the live Render URL goes in its README. Due date on Canvas.

The key, one more time. The environment variable on your Render app must be the **new class key**. An app built on the old key shows a 401 error in its Logs and earns 0 of the live points until the variable is replaced.

Before you leave: knit, then commit and push `markdowns/class07.Rmd`, `class07.html`, and `apps/job_scout_chat/`. If it is not pushed, it is not done.

Ungraded Practice Questions

1. Which five states have the highest **remote share** among states with at least 50 postings?

2. How many postings mention "SQL" in the `title`? Hint: `grepl("SQL", title)` inside `filter()`.

3. For each `category`, the earliest and latest `posted_at`.

4. Add `region` with the join, then count remote postings per region.

5. On the Airbnb data: superhosts (`host_is_superhost == "t"`) versus the rest, median price per city.

▶ **Answers**

Appendices

Not covered live. A1 for the bracket-to-verb translation, A2 when a deploy fails, A3 for the reading.

A1: Base R Equivalents

```r
# Class 04 spelling
jobs[jobs$remote == 1, ]
jobs[, c("company", "title")]
jobs[order(-jobs$days_open), ]
jobs$days_open = as.numeric(
  as.Date(jobs$last_seen_at) -
  as.Date(jobs$posted_at))
table(jobs$category)
merge(by_state, states,
      by = "location_state",
      all.x = TRUE)
```

```r
# today's spelling
# jobs |> dplyr::filter(rem
# jobs |> dplyr::select(com
# jobs |> dplyr::arrange(dp
# jobs |> dplyr::mutate(day
#   as.numeric(as.Date(last
#              as.Date(post
# jobs |> dplyr::count(cate
# by_state |>
#   dplyr::left_join(states
#                    by = "
```

What changed, and what did not

Every bracket line names the table twice (`jobs[jobs$...]`); the verb names it once and knows the columns. The trailing comma, the `-` for descending, and `all.x = TRUE` are replaced by words.

The results are the same data frames. Base R is not wrong; it is harder to read aloud, and reading aloud is how you check a chain.

Where brackets remain the right tool: one element from a vector or a list, `x[[1]]` and `campus$Farmer`. dplyr is for tables.

A2: Deployment Troubleshooting

Logs say `401` or `Incorrect API key`

The environment variable is missing, misspelled, or holds the old class key. On the service page open **Environment**: the key must be exactly `OPENAI_API_KEY`, the value the new key. Saving it redeploys the app.

Build fails at once: `Dockerfile not found`

The **Root Directory** is empty or wrong, so Render looked at the top of the repository. Settings → Build & Deploy → Root Directory = `apps/job_scout_chat` (or the Assignment 05 folder), then Manual Deploy.

Build fails later in the log

Read the last twenty lines of **Logs**. A package name in the error means the Dockerfile's `install.packages()` line was edited; restore it from the zip. A line-ending error (`\r`) means the Dockerfile was saved by a Windows editor without the `.gitattributes` rules; re-upload the original.

Live, but the page hangs or shows 502

A free app sleeps after 15 idle minutes; the first visit wakes it in about a minute, and a 502 in between is normal. If it never answers, the Dockerfile must listen on the port Render assigns: keep the class `CMD` line with `Sys.getenv('PORT', '7860')`. Locally, a stale `.Renviron` key needs Session → Restart R.

Where to look first, always: the **Logs** tab of the service, then **Events** for which commit deployed. The last lines name the file and the line that failed. Copy them into your question if you ask for help.

Sources: Render: troubleshooting deploys; Render: environment variables and secrets; Render: free instances; R startup: .Renviron.

A3: Reading

The two chapters that cover today

R for Data Science, Data transformation: the six verbs on the flights data, with the same "rows, columns, groups" organization as today's verb table.

R for Data Science, Joins: keys, mutating joins, filtering joins, with the diagrams that today's animation borrowed its colors from.

For the desk

dplyr cheatsheet: one page, every verb, print it.

dplyr: Two-table verbs: the joins, with the `by = ` spellings for keys that have different names.

dplyr reference: when you need the exact arguments.

Not covered today, on purpose: `pivot_longer()` and `pivot_wider()` (tidy data, Oct 7), string functions (`stringr`, with scraping), and window functions inside `mutate()`. Each has a class.